TRANSFORMATION · DOMAIN 3 - DATA TRANSFORMATION, CLEANSING, QUALITY

Shuffle, skew and broadcast

Section 3. What 'efficient' means once the query is running.

SECTION 3 of 10 WEIGHT 10% TOPIC Execution

admin@sthithapragna.com

Every expensive query has a shuffle in it.
The question is never whether it shuffles, but how much and how evenly.

admin@sthithapragna.com

Narrow and wide

- **Narrow:** each output partition depends on one input partition. Filters, projections, casts.
- **Wide:** output partitions depend on many input partitions. Joins, group by, distinct, sorts, window functions.
- A wide transformation forces a **shuffle**, and a shuffle forces a **stage boundary**.
- Counting stage boundaries in a plan is a quick estimate of how much reorganisation a query needs.

> ON THE EXAM
>
> Filtering before a join is not a style preference. It reduces what has to cross the network, which is the expensive part.

Four strategies, and when each applies

STRATEGY	CHARACTER
Broadcast hash	Copies the small side to every executor. The large side never moves.
Sort merge	Shuffles both sides, sorts, then merges. The default for large-to-large.
Shuffle hash	Shuffles both sides and builds a hash table instead of sorting.
Broadcast nested loop	For non-equi joins and cross joins, where there is no key to hash on.

ON THE EXAM

Seeing a broadcast nested loop join over large data in a plan is a warning rather than a win. It usually means an equality condition went missing.

The fast join that can take out the driver

- No shuffle of the large side at all. When one side truly fits, nothing else comes close.
- The small side is **collected to the driver**, then sent to every executor.
- A **BROADCAST hint is honoured even when the table exceeds the threshold**. That is how a hint causes an out-of-memory failure.
- So a hint is a statement about what you believe, and belief about table size ages badly.

> ON THE EXAM
>
> A broadcast that worked for a year and then failed is usually a dimension table that grew past what the driver can hold.

Four things AQE does at runtime

FEATURE	WHAT IT FIXES
Switch sort merge to broadcast	A join planned as large-to-large where one side turns out small.
Coalesce partitions	Hundreds of tiny partitions after a shuffle, each with per-task overhead.
Handle skew join	One partition far larger than the others, holding up the whole stage.
Propagate empty relations	A branch of the plan that turns out to have no rows at all.

ON THE EXAM

AQE is enabled by default. It works from real statistics gathered at the end of a shuffle, which is why it beats planning from stale table statistics.

Numbers worth knowing

SETTING	DEFAULT
Shuffle partitions	200. Set it to auto for auto-optimized shuffle.
Runtime broadcast threshold	30 MB, for AQE switching a join at runtime.
Advisory partition size	64 MB, the target size after coalescing.
Partition coalescing	Enabled.

ON THE EXAM

Two hundred shuffle partitions is a fixed default that suits neither a small dataset nor a huge one. Coalescing exists largely because of it.

Shuffle partitions and streaming checkpoints

- In Structured Streaming, **shuffle partitions cannot be changed** between restarts from the same checkpoint.
- So a tuning change that is routine in batch is a **checkpoint-invalidating change** in a stream.
- Decide the value before the stream goes live, because changing it later means a new checkpoint.
- This is the same principle as module 6: anything that changes the shape of the state cannot be carried forward.

> ON THE EXAM
>
> A perfectly reasonable batch optimisation becomes a breaking change the moment the workload is streaming.

admin@sthithapragna.com

How the engine decides a partition is skewed

- A partition is skewed when **both** tests pass: it is larger than a **multiple of the median**, and larger than an **absolute size threshold**.
- Requiring both stops the engine treating a slightly uneven but small partition as a problem.
- When detected, the skewed partition is split so the work spreads across more tasks.
- The symptom in the interface is one task running far longer than every other task in the stage.

> ON THE EXAM
>
> Skew is why a stage can be 99% complete for twenty minutes. The median task finished long ago; one is still going.

Where AQE skew handling does not reach

- It applies **only to shuffle-based joins**: sort merge and shuffle hash.
- **Broadcast joins are never skew-optimized.**
- In a **LEFT OUTER JOIN, only left-side skew** can be optimized.
- A **FULL OUTER JOIN is not skew-optimized on either side.**

> ON THE EXAM
>
> These limits are exactly the sort of specific fact a professional paper tests. AQE is not a universal answer to skew.

Salting, and its cost

- Add a random component to the join key so one hot value spreads across many partitions.
- The other side must be **expanded to match** every salt value, which multiplies its rows.
- It changes the query, which means it changes what a reader has to understand.
- Reach for it when AQE cannot help: a skewed full outer join, or skew outside a join entirely.

> ON THE EXAM
>
> Salting is the manual version of what AQE does automatically. Try the automatic route first, and know why it failed before adding complexity.

Four join hints, in priority order

HINT	MEANING
BROADCAST	Broadcast this side. Highest priority. Aliases: BROADCASTJOIN, MAPJOIN.
MERGE	Use sort merge. Aliases: SHUFFLE_MERGE, MERGEJOIN.
SHUFFLE_HASH	Use shuffle hash. Smaller side becomes the build side.
SHUFFLE_REPLICATE_NL	Shuffle and replicate nested loop. Lowest priority.

ON THE EXAM

When both sides carry different hints, that order decides. And a hint is a suggestion: a strategy that cannot support the join type will not be used.

The range join hint

- A join on a **range condition** has no equality key, so the engine falls back to nested loops.
- The **range join hint** bins the range values so a hash-based lookup becomes possible.
- That turns something close to a Cartesian product into a manageable join.
- Typical shape: an event timestamp falling between a start and an end column.

> ON THE EXAM
>
> If a plan shows a broadcast nested loop join over large inputs and the condition is a range, this hint is what the question is pointing at.

How to tell AQE actually did something

- Compare the **initial plan** against the **final plan**. Different physical join nodes mean a strategy switched.
- Coalescing shows as a custom shuffle reader marked **Coalesced**.
- Skew handling shows as a sort merge join with an **is skew** field set to true.
- An empty relation shows as part of the plan replaced by a local table scan with nothing in it.

> ON THE EXAM
>
> When a question asks how you would confirm an optimisation happened, these markers are the evidence rather than the runtime.

What to carry into the exam

- Wide transformations shuffle. Filtering before a join reduces what crosses the network.
- **AQE is on by default**: switches joins, coalesces partitions, handles skew, prunes empty relations.
- Skew handling covers **shuffle joins only**. Never broadcast, and never a full outer join.
- A **BROADCAST hint overrides the threshold**, which is how a hint takes out the driver.
- **Shuffle partitions cannot change** between streaming restarts on the same checkpoint.

> ON THE EXAM
>
> Before tuning anything, check whether AQE already handled it. Most manual shuffle tuning advice predates it being on by default.

A nightly job joins a 4 TB transactions table to a country reference table. It has run in eight minutes for a year. Last week it began failing with a driver out-of-memory error.

The query contains a BROADCAST hint on the reference table, added when the job was written. The reference table has grown as new markets were added.

What is the cause?

A The transactions table has grown past what a broadcast join can handle.

B The hint is honoured even above the broadcast threshold, so the grown reference table is collected to the driver.

C AQE has stopped switching the join to broadcast because statistics are stale.

D Shuffle partitions must be increased to accommodate the larger reference table.

◆ PAUSE AND DECIDE

What is the cause?

A The transactions table has grown past what a broadcast join can handle.

The broadcast side is the small one. The large side is never collected.

B The hint is honoured even above the broadcast threshold, so the grown reference table is collected to the driver.

A hint overrides the automatic size check, which is exactly what makes it dangerous as data grows.

C AQE has stopped switching the join to broadcast because statistics are stale.

The hint forces the broadcast. AQE is not making this decision.

D Shuffle partitions must be increased to accommodate the larger reference table.

A broadcast join does not shuffle the large side. Partition count is not involved.

ON THE EXAM

Removing the hint lets AQE decide from real statistics, and it will simply stop broadcasting once the table is too big.

> A full outer join between two large tables runs for hours. The Spark UI shows one task in the join stage running far longer than the rest, with the stage stuck near completion. One join key value accounts for roughly 40% of rows on both sides.
>
> An engineer says AQE will handle the skew automatically.

What is the correct assessment?

A They are right. AQE splits skewed partitions in any shuffle-based join.

B A FULL OUTER JOIN is not skew-optimized on either side, so a manual approach such as salting is needed.

C AQE will handle it once statistics are refreshed with ANALYZE.

D The join should be broadcast, which avoids skew entirely.

◆ PAUSE AND DECIDE

What is the correct assessment?

A They are right. AQE splits skewed partitions in any shuffle-based join.

Nearly right, and the join type limits apply. Full outer is the exception.

B A FULL OUTER JOIN is not skew-optimized on either side, so a manual approach such as salting is needed.

A documented limit. Adaptive execution will not rescue this particular shape.

C AQE will handle it once statistics are refreshed with ANALYZE.

AQE works from runtime statistics precisely so it does not depend on collected ones.

D The join should be broadcast, which avoids skew entirely.

Both sides are large, and broadcast joins are never skew-optimized anyway.

ON THE EXAM

A is the answer most people would give, and it is right about the mechanism and wrong about this join type. Read the join type first.

> A batch job was tuned by reducing shuffle partitions from 200 to 32, which cut its runtime substantially. An engineer applies the same change to a Structured Streaming job that shares the same logic.
>
> The streaming job fails on restart.

Why?

A 32 partitions is below the minimum permitted for streaming queries.

B Shuffle partitions cannot be changed between restarts from the same checkpoint in Structured Streaming.

C Streaming requires shuffle partitions to match the number of source partitions.

D AQE must be disabled before shuffle partitions can be set manually.

◆ PAUSE AND DECIDE

Why?

A 32 partitions is below the minimum permitted for streaming queries.

No such minimum exists.

B Shuffle partitions cannot be changed between restarts from the same checkpoint in Structured Streaming.

Documented explicitly. A routine batch tuning change invalidates a streaming checkpoint.

C Streaming requires shuffle partitions to match the number of source partitions.

Invented requirement.

D AQE must be disabled before shuffle partitions can be set manually.

Both can be set together. AQE coalesces from whatever initial number is configured.

ON THE EXAM

The same edit is routine tuning in batch and a breaking change in a stream. That asymmetry is what the question is testing.

> A query joins an events table to a promotions table on the condition that the event timestamp falls between the promotion's start and end times. There is no equality condition.
>
> The plan shows a broadcast nested loop join and the query has never completed on full data.

What should the engineer do?

A Add a BROADCAST hint on the promotions table to force a hash join.

B Use a range join hint so the range values are binned and hashed.

C Increase shuffle partitions so the nested loop is spread over more tasks.

D Rewrite it as a cross join with a WHERE clause, which the optimiser handles better.

◆ PAUSE AND DECIDE

What should the engineer do?

A Add a BROADCAST hint on the promotions table to force a hash join.

A hash join needs an equality key. Broadcasting does not create one.

B Use a range join hint so the range values are binned and hashed.

Bins the range so a hash lookup replaces the nested loop. Built for exactly this shape.

C Increase shuffle partitions so the nested loop is spread over more tasks.

More tasks doing quadratic work is still quadratic work.

D Rewrite it as a cross join with a WHERE clause, which the optimiser handles better.

That is what the query already is. Writing it explicitly changes nothing.

ON THE EXAM

The absence of an equality condition is the signal. Without a key to hash on, the engine has nothing to organise the join around.

> A team inherited a job containing a long list of Spark configuration settings, added over several years. It sets shuffle partitions to 1000, disables adaptive query execution, and manually specifies broadcast thresholds.
>
> The job is slow and the team wants a starting point for improving it.

What should they try first?

A. Increase shuffle partitions further, since 1000 is clearly insufficient for a slow job.

B. Re-enable AQE and remove the manual settings, then measure before tuning anything by hand.

C. Add BROADCAST hints to every join so no join has to shuffle.

D. Salt all join keys pre-emptively to eliminate any possible skew.

◆ PAUSE AND DECIDE

What should they try first?

A Increase shuffle partitions further, since 1000 is clearly insufficient for a slow job.

More partitions means more tasks and more overhead. Nothing suggests too few.

B Re-enable AQE and remove the manual settings, then measure before tuning anything by hand.

Most of that tuning predates AQE being on by default, and disabling it removes four automatic optimisations.

C Add BROADCAST hints to every join so no join has to shuffle.

Broadcasting a large side is how you take out the driver.

D Salt all join keys pre-emptively to eliminate any possible skew.

Adds complexity everywhere to solve a problem nobody has confirmed exists.

ON THE EXAM

Accumulated configuration is often the cause rather than the cure. Removing it and measuring is a legitimate first step.

> An engineer is asked to prove that a query optimisation actually took effect after enabling adaptive query execution. The query now runs faster, but the change coincided with a cluster upgrade.
>
> They need evidence that AQE specifically did something.

What should they look for?

A. A shorter total runtime in the job history, compared against the previous week.

B. Differences between the initial and final plans, plus markers such as a coalesced shuffle reader or a join with is-skew set.

C. A lower DBU count in the billable usage system table.

D. Fewer files written to the target table.

◆ PAUSE AND DECIDE

What should they look for?

A. A shorter total runtime in the job history, compared against the previous week.
Confounded by the cluster upgrade, which is exactly why the question mentions it.

B. Differences between the initial and final plans, plus markers such as a coalesced shuffle reader or a join with is-skew set.
AQE rewrites between the initial and final plan, and each optimisation leaves a specific marker.

C. A lower DBU count in the billable usage system table.
Also confounded, and it measures cost rather than which optimisation fired.

D. Fewer files written to the target table.
AQE re-plans the query. It is not a file-layout feature.

ON THE EXAM

Runtime tells you something improved. The plan tells you what. When a question mentions a confounding change, it is asking for the plan.